CSRankingsArena: An Agentic Research Paper Competition Web Platform

Devanathan Nallur Gandamani
Master's of Science Department of Computer Science
University of California - Santa Cruz
Student ID : 2086936
Email: dnallurg@ucsc.edu

***Abstract*—In this paper, we present CSRankingsArena, a novel web-based platform that creates virtual conferences for computer science research papers sourced from repositories like arXiv. Our system implements an autonomous research evaluation framework where AI agents serve as reviewers in a tournament-style competition, eliminating human bias from the assessment process. Papers compete in head-to-head matches across various categories (AI, Computer Architecture, Operating Systems) and subcategories (e.g., Vision, ML, LLMs), with rankings determined by win-loss records. The platform incorporates comprehensive match data, agent-provided reviews with multi-dimensional scoring metrics, and detailed comparison rationales. We demonstrate the system's capabilities using a case study of AI vision papers from 2025, showing how the tournament structure effectively identifies strong research contributions while providing transparent evaluation criteria. CS RankingsArena addresses key challenges in traditional peer review by offering scalability, consistency, and reduced reviewer bias, while still allowing for human oversight through a community feedback mechanism on agent evaluations. Our evaluation shows the system successfully processes complex research content and produces coherent, justifiable rankings that can supplement traditional publication venues.**

***Index Terms*—research evaluation, artificial intelligence, large language models, peer review, tournament structure, academic ranking**

I. Introduction

The academic peer review process, while foundational to scientific progress, faces significant challenges in the modern research landscape. Traditional peer review suffers from several limitations: scalability issues as submission volumes grow, inconsistency in reviewer standards, potential reviewer bias, and time delays that slow scientific progress [1]. These challenges are particularly acute in rapidly evolving fields like computer science, where the volume of research output continues to accelerate.

Recent advances in large language models (LLMs) and AI agents present an opportunity to reimagine the research evaluation process. These systems can process vast amounts of information, maintain consistent evaluation criteria, and operate without human biases that might influence traditional reviews. However, simply replacing human reviewers with AI agents introduces new challenges, including how to structure the evaluation process, ensure meaningful assessments, and maintain human oversight of the system.

We introduce CS RankingsArena, a novel platform that leverages AI agents to create a competitive tournament structure for evaluating research papers. Our system represents a departure from traditional peer review, implementing a sports-inspired "league" model where papers compete in head-to-head matches evaluated by AI agents. This approach offers several advantages:

1) **Scalability**: The system can process large volumes of papers without human reviewer limitations
2) **Consistency**: Agents apply the same evaluation criteria across all papers
3) **Reduced bias**: The tournament structure eliminates some forms of human bias
4) **Transparency**: All evaluations and comparisons are available for community scrutiny
5) **Community feedback**: Users can provide input on agent evaluations, creating a hybrid human-AI system

The CS RankingsArena platform organizes papers by research categories (e.g., AI, Computer Architecture, Operating Systems) and subcategories (e.g., Vision, ML, LLMs for AI), creating focused competitions within research domains. Papers are matched in pairs, with AI agents serving as reviewers who evaluate each paper on multiple dimensions and determine a winner based on comprehensive analysis.

This paper presents the design, implementation, and evaluation of CS RankingsArena. We discuss the system architecture, the tournament structure, the agent evaluation process, and the user interface for browsing papers and match results. We evaluate the system using a dataset of AI vision papers from 2025, analyzing the quality of agent evaluations and the resulting paper rankings.

Our contributions include:

1) An architecture for agentic research paper evaluation using a tournament structure
2) Methods for structuring head-to-head paper comparisons across multiple evaluation dimensions
3) A web-based platform for browsing papers, matches, and rankings with community feedback
4) Analysis of system performance on real-world research papers

CS RankingsArena represents a step toward reimagining research evaluation for the modern era, combining the strengths

of AI agents with human oversight to create a more scalable, consistent, and transparent process.

II. System Design

A. Architecture Overview

CS RankingsArena's architecture consists of four main components: the data ingestion pipeline, the match generation system, the agent evaluation framework, and the web interface. Figure 1 illustrates this architecture.

The system implements a client-server architecture with a React frontend, Node.js backend, and Postgres database. The backend exposes RESTful APIs that handle paper data management, match processing, and user interactions. The architecture follows modern software engineering principles of separation of concerns, modularity, and stateless API design.

```
CSRankingsArena/
        frontend/           # React UI
              src/
                  components/ # UI components
                  pages/      # Page components
                  services/   # API integration
                  utils/      # Helper functions
        backend/            # Node.js server
              controllers/    # Request handlers
              models/         # Data models
              routes/         # API endpoints
              services/       # Business logic
        papers/             # Paper storage
              ai/
                    vision/2025/
                    ml/2025/
                    llms/2025/
              systems/
        scripts/            # Processing
              arxiv-scraper/
              match-generator/
              database-seeder/
        agents/             # AI agents
              claude/
              gpt/
              evaluators/
        database/           # DB files
            migrations/
            seeds/
```

Listing 1: CS RankingsArena project directory structure

The data ingestion pipeline scrapes research repositories like arXiv to extract paper metadata and full-text content. Papers are categorized by research domain and stored in a structured format (YAML) that includes title, abstract, authors, publication year, URL, category, and subcategory.

```
router.get('/api/papers', async (req, res) => {
  try {
    const { category, subcategory, year } = req.
        query;
    const query = {};

    if (category) query.category = category;
    if (subcategory) query.subcategory = subcategory
        ;
    if (year) query.year = parseInt(year);

    const papers = await Paper.find(query).sort({
        title: 1 });
    res.json(papers);
  } catch (error) {
    console.error('Error fetching papers:', error);
    res.status(500).json({ error: 'Failed to fetch
        papers' });
  }
});
```

Listing 2: Paper data retrieval API endpoint

The match generation system creates pairwise matches between papers within the same category and subcategory, ensuring a comprehensive tournament structure. Each paper competes against multiple others to establish a robust ranking.

```
function generateMatches(papers, category,
    subcategory, year) {
  const matches = [];

  // Create all possible paper pairs
  for (let i = 0; i < papers.length; i++) {
    for (let j = i + 1; j < papers.length; j++) {
      const matchId = `match-${category}-${
          subcategory}-${year}-${papers[i].id}-${
          papers[j].id}`;

      matches.push({
        id: matchId,
        paperIds: [papers[i].id, papers[j].id],
        category,
        subcategory,
        year,
        createdAt: new Date().toISOString(),
        status: 'pending',
        reviews: [],
        comparison: null
      });
    }
  }

  return matches;
}
```

Listing 3: Match generation logic

The agent evaluation framework leverages multiple LLM-based agents (Claude, GPT-4, Gemini, Llama) to review papers independently and generate head-to-head comparisons. Agents evaluate papers on five dimensions:

1) Technical depth
2) Novelty
3) Experimental validation
4) Clarity of presentation
5) Potential impact

Each dimension receives a numerical score (0-10) and a brief comment explaining the rating. Agents also provide an overall score, summary, and detailed feedback for each paper. For head-to-head comparisons, agents determine a winner, explain their reasoning, and compare scores across all dimensions.

```
async function evaluatePaper(paperId, agentId) {
  try {
    const paper = await Paper.findById(paperId);
    if (!paper) throw new Error(`Paper ${paperId}
        not found`);

    const promptTemplate = `
      You are an expert reviewer evaluating a
          computer science research paper.
      Please provide scores (0-10) and brief
          comments for the following dimensions:

      1. Technical depth
      2. Novelty
      3. Experimental validation
      4. Clarity of presentation
      5. Potential impact
```

```

      Also provide an overall score, summary, and detailed feedback.

      Paper title: ${paper.title}
      Paper abstract: ${paper.abstract}
      Authors: ${paper.authors.join(', ')}
    `;

    const response = await callAgent(agentId, promptTemplate);
    return parseAgentResponse(response);
  } catch (error) {
    console.error(`Error in paper evaluation: ${error}`);
    throw error;
  }
}
```

Listing 4: Agent evaluation service example

The web interface allows users to browse papers by category, subcategory, and year, view individual paper details, explore match results, and provide feedback on agent evaluations through comments and like/dislike interactions.

B. Data Structure

CS RankingsArena uses a structured data format to store paper information and match results. Paper data is organized in YAML files with the following structure:

```yaml
papers:
  - id: [unique identifier]
    title: [paper title]
    abstract: [paper abstract]
    authors: [list of authors]
    year: [publication year]
    url: [source URL]
    pdfUrl: [PDF URL]
    category: [main category]
    subcategory: [subcategory]
    venue: [publication venue]
    match_opponents: [list of papers this paper competed against]
```

Listing 5: Paper data structure

Match data is stored in a separate YAML structure:

```yaml
matches:
  - id: [unique match identifier]
    paperIds: [list of two competing paper IDs]
    category: [category]
    subcategory: [subcategory]
    year: [year]
    createdAt: [timestamp]
    status: [match status]
    reviews: [agent reviews for each paper]
    comparison:
      winner: [winning paper ID]
      reasoning: [explanation for the decision]
      overallScores: [overall scores]
      metricComparison: [dimension-by-dimension comparison]
```

Listing 6: Match data structure

This structured data format enables efficient processing, analysis, and presentation of paper competitions.

C. Tournament Structure

CS RankingsArena implements a round-robin tournament structure where each paper competes against multiple others within its category and subcategory. This structure ensures that rankings are not determined by a single evaluation but by performance across multiple matches.

The match generation algorithm creates matches by:

1) Grouping papers by category, subcategory, and year
2) Creating all possible paper pairs within each group
3) Assigning unique match IDs and initializing match structures
4) Randomly assigning different agent reviewers to each match

This approach creates a comprehensive competition structure while distributing evaluation responsibilities across different agents, reducing the impact of any individual agent's biases.

D. Agent Evaluation Process

The agent evaluation process consists of three phases:

1) **Individual paper review**: Agents analyze each paper independently, providing scores and feedback on five dimensions. This evaluation includes an overall score, a summary of strengths and weaknesses, and detailed feedback on specific aspects.
2) **Comparative analysis**: Agents compare the two papers directly, identifying relative strengths and weaknesses across all evaluation dimensions.
3) **Winner determination**: Based on both individual and comparative analyses, agents determine a winner and provide a detailed explanation for their decision.

Agents are provided with specific evaluation criteria for each dimension:

- **Technical depth**: Rigor of methodology, theoretical foundations, and analytical approach
- **Novelty**: Originality of ideas, approaches, or findings compared to existing work
- **Experimental validation**: Quality and comprehensiveness of empirical evaluation
- **Clarity of presentation**: Organization, writing quality, and accessibility of ideas
- **Potential impact**: Likely influence on the field and practical applications

This structured evaluation process ensures comprehensive assessment while providing transparency into the decision-making process.

E. Web Interface

The CS RankingsArena web interface is designed to provide an intuitive browsing experience for papers, matches, and rankings. The interface includes:

1) **Homepage**: Overview of the platform with navigation to different categories
2) **Category pages**: Lists of subcategories within each main research area

3) **Subcategory pages**: Papers organized by year with ranking information
4) **Paper details**: Comprehensive information about individual papers
5) **Match pages**: Results of head-to-head comparisons with agent evaluations
6) **Leaderboard**: Rankings based on win-loss records within each subcategory
7) **Feedback mechanism**: User comments and like/dislike interactions on matches

The interface uses a responsive design that adapts to different screen sizes, ensuring accessibility across devices. Visual elements such as charts and graphs illustrate score comparisons and rankings, enhancing the user experience.

III. Platform Features

Building upon the core architecture described in previous sections, CS RankingsArena incorporates several advanced features that extend its capabilities beyond basic paper evaluation and ranking.

A. Automated Paper Generation and Processing Pipeline

The system includes a sophisticated batch paper generation capability that automates the entire paper ingestion workflow:

1) Batch Paper Generation: The platform can generate batches of papers following predefined YAML structures, ensuring consistency across all imported content:

```yaml
papers:
- abstract:
  keywords:
  references:
  title:
  url:
```

Listing 7: Enhanced paper YAML structure with batch generation support

2) Server-Side YAML Processing: The backend implements intelligent YAML processing with dynamic routing capabilities:

```
async function processPaperBatch(yamlFile) {
  try {
    const yamlContent = yaml.load(fs.readFileSync(
        yamlFile, 'utf8'));
    const processedPapers = [];

    for (const paper of yamlContent.papers) {
      // Extract category, subcategory, and year
          from content
      const { category, subcategory, year } = await
          extractMetadata(paper);

      // Validate paper structure
      const validationResult = await
          validatePaperStructure(paper);

      if (validationResult.isValid) {
        processedPapers.push({
          ...paper,
          category,
          subcategory,
          year,
          processed_at: new Date().toISOString()
        });
      }
    }

    return processedPapers;
  } catch (error) {
    console.error('Error processing paper batch:',
        error);
    throw error;
  }
}
```

Listing 8: Server-side YAML processing with dynamic category extraction

B. Intelligent Taxonomy Management

1) Slug-Based Architecture and Database Seeding: Categories and subcategories are converted to URL-friendly slugs and can be seeded into the database with a single command:

```
async function seedCategories() {
  const discoveredTaxonomy = await
      discoverCategories(allPapers);

  for (const category of discoveredTaxonomy.
      categories) {
    const categorySlug = slugify(category);

    await db('categories').insert({
      name: category,
      slug: categorySlug,
      created_at: new Date()
    }).onConflict('slug').ignore();

    for (const subcategory of discoveredTaxonomy.
        subcategories[category]) {
      const subcategorySlug = slugify(subcategory);

      await db('subcategories').insert({
        category_slug: categorySlug,
        name: subcategory,
        slug: subcategorySlug,
        created_at: new Date()
      }).onConflict(['category_slug', 'slug']).
          ignore();
    }
  }
}
```

Listing 9: Database seeding script for categories

C. AI-Powered Match Generation with Multi-Agent Support

Building on the paper's agent integration, your system provides:

- **Multi-Agent Connectivity**: Direct integration with leading AI services (Claude, GPT-4) for paper evaluation
- **Mock Match Generation**: Ability to create realistic paper comparisons for testing and demonstration purposes
- **Automated Evaluation Pipeline**: Seamless generation of detailed comparison results with scoring across multiple dimensions

The system extends the basic agent integration to support comprehensive match generation:

```
async function generateAgentMatches(papers, agents =
    ['claude', 'gpt4']) {
  const matches = [];
```

```

  for (let i = 0; i < papers.length; i++) {
    for (let j = i + 1; j < papers.length; j++) {
      const matchId = generateMatchId(papers[i], papers[j]);
      const selectedAgent = agents[Math.floor(Math.random() * agents.length)];

      // Generate mock match using selected agent
      const matchResult = await generateMockMatch(
        papers[i],
        papers[j],
        selectedAgent
      );

      matches.push({
        id: matchId,
        paper1: papers[i].id,
        paper2: papers[j].id,
        agent_evaluator: selectedAgent,
        evaluation_result: matchResult,
        status: 'completed',
        created_at: new Date().toISOString()
      });
    }
  }

  return matches;
}
```

Listing 10: Enhanced multi-agent match generation

D. Advanced Leaderboard and Ranking System

1) Multi-Dimensional Leaderboard: The leaderboard system provides comprehensive ranking analysis:

```
function calculateAdvancedLeaderboard(matches, subcategory, year) {
  const paperStats = new Map();

  // Calculate comprehensive statistics
  matches.forEach(match => {
    [match.paper1, match.paper2].forEach(paperId => {
      if (!paperStats.has(paperId)) {
        paperStats.set(paperId, {
          wins: 0, losses: 0, draws: 0,
          totalScore: 0, matchCount: 0,
          winStreak: 0, currentForm: [],
          opponentStrengths: []
        });
      }
    });

    // Update match statistics
    updateMatchStatistics(paperStats, match);
  });

  // Generate ranking with multiple metrics
  return Array.from(paperStats.entries())
    .map(([paperId, stats]) => ({
      paperId,
      points: stats.wins * 3 + stats.draws * 1,
      winRate: (stats.wins / stats.matchCount) * 100,
      avgScore: stats.totalScore / stats.matchCount,
      form: stats.currentForm.slice(-5),
      strengthOfSchedule: calculateSOS(stats.opponentStrengths)
    }))
    .sort((a, b) => b.points - a.points || b.winRate - a.winRate);
}
```

Listing 11: Advanced leaderboard calculation

E. Community Engagement and Authentication

1) GitHub-Integrated User Interactions: The platform implements secure user authentication through GitHub:

```
async function handleUserInteraction(matchId, userId, interactionType, content) {
  // Verify GitHub authentication
  const user = await verifyGitHubAuth(userId);
  if (!user.verified) {
    throw new Error('GitHub authentication required');
  }

  const interaction = {
    match_id: matchId,
    user_id: userId,
    github_username: user.username,
    type: interactionType, // 'like', 'dislike', 'comment'
    content: content,
    is_verified: true,
    created_at: new Date()
  };

  await db('match_interactions').insert(interaction);

  // Update match engagement metrics
  await updateMatchEngagement(matchId);

  return interaction;
}
```

Listing 12: GitHub-authenticated user interaction system

2) Integrated Issue Management: Users can raise issues directly on the project's GitHub repository.

F. Paper Quality Assurance System

1) Automated Disqualification Detection: The system can identify papers that may not meet quality standards.

2) Community Moderation: Users can provide input on disqualification decisions:

G. Comprehensive Search Functionality

The global search feature provides powerful discovery capabilities:

- **Multi-field Search**: Searches across authors, titles, and abstracts.
- **Cross-category Discovery**: Users can find relevant papers regardless of categorical boundaries
- **Research Accessibility**: Makes the vast paper database easily navigable for researchers.

The platform implements intelligent search across multiple paper attributes:

```
async function performGlobalSearch(query, options = {}) {
  const searchFields = ['title', 'abstract', 'authors'];
  const searchQuery = db('papers')
    .select('*')
    .select(db.raw('ts_rank_cd(search_vector, plainto_tsquery(?)) as rank', [query]))
    .where(function() {
      // Full-text search across multiple fields
      this.whereRaw('search_vector @@ plainto_tsquery(?)', [query])
        .orWhere('title', 'ilike', `%${query}%`)
        .orWhere('abstract', 'ilike', `%${query}%`)
        .orWhereRaw('? = ANY(authors)', [query]);
    });

  // Apply filters if provided
  if (options.category) {
    searchQuery.where('category', options.category);
  }

  if (options.year) {
    searchQuery.where('year', options.year);
  }

  // Order by relevance and recency
  searchQuery.orderBy('rank', 'desc')
    .orderBy('year', 'desc')
    .limit(options.limit || 50);

  const results = await searchQuery;

  // Enhance results with match statistics
  return await enhanceSearchResults(results);
}
```

Listing 13: Multi-field intelligent search system

H. Data Integration and Export Capabilities

The system maintains comprehensive data export functionality:

```
async function exportComprehensiveData(category, subcategory, year) {
  const papers = await getPapers({ category, subcategory, year });
  const matches = await getMatches({ category, subcategory, year });
  const interactions = await getUserInteractions(matches.map(m => m.id));
  const qualityAssessments = await getQualityAssessments(papers.map(p => p.id));

  const exportData = {
    metadata: {
      export_timestamp: new Date().toISOString(),
      category,
      subcategory,
      year,
      total_papers: papers.length,
      total_matches: matches.length,
      total_interactions: interactions.length
    },
    papers: papers.map(paper => ({
      ...paper,
      quality_assessment: qualityAssessments.find(qa => qa.paper_id === paper.id),
      match_record: calculateMatchRecord(paper.id, matches)
    })),
    matches: matches.map(match => ({
      ...match,
      community_interactions: interactions.filter(i => i.match_id === match.id),
      engagement_metrics: calculateEngagementMetrics(match.id, interactions)
    })),
    summary_statistics: generateSummaryStatistics(papers, matches, interactions)
  };

  return yaml.dump(exportData);
}
```

Listing 14: Enhanced data export with user interactions

I. System Integration and Workflow

These features work together to create a comprehensive research evaluation ecosystem:

1) **Data Flow**: Papers → YAML Processing → Categorization → Database Seeding → Match Generation → Community Interaction
2) **Quality Assurance**: Automated quality checks → Community validation → Transparent discussion → Continuous improvement
3) **User Experience**: Easy discovery (search) → Detailed evaluation (matches) → Community engagement (feedback) → System improvement (issues)

The CSRankingsArena represents a significant advancement over traditional peer review systems by combining AI automation with community oversight, creating a scalable, transparent, and continuously improving research evaluation platform. The integration of these features addresses key challenges in academic paper discovery, evaluation, and community engagement while maintaining the transparency and rigor essential for scientific progress. “‘

IV. System Architecture

A. Data Structure

The system employs a hierarchical data structure stored in YAML format, consisting of two primary components:

1) Paper Data: Paper information is stored in category-specific YAML files (e.g., `ai-vision-2025-papers.yaml`), containing:

- Paper metadata (ID, title, authors, year)
- Abstract and content information
- Publication venue and URLs
- Category and subcategory classification

2) Match Data: Match information is stored in corresponding YAML files (e.g., `ai-vision-2025-matches.yaml`), containing:

- Match identification and metadata
- Performance metrics (wins, losses, draws)
- Statistical data (points, win rate, rating differences)
- Match history and opponent information

B. Database Schema

The system utilizes a relational database to store user interactions, with the following key tables:

1) Match Interactions Table:

```sql
CREATE TABLE match_interactions (
    id SERIAL PRIMARY KEY,
    match_id VARCHAR NOT NULL,
    user_id VARCHAR NOT NULL,
    type VARCHAR NOT NULL,
    content TEXT,
    is_anonymous BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

V. System Architecture & Design

CS RankingsArena implements a modern three-tier architecture designed to support the paper evaluation and tournament workflow. This section provides a detailed overview of the system design, data flow, and component interactions.

A. High-Level Architecture

The system comprises three principal layers: a React-based frontend, a Node.js backend with Hapi server, and a PostgreSQL database, as illustrated in Figure 1.

The frontend layer handles user interactions and presentation, while the backend implements business logic and API endpoints. The database layer ensures persistent storage of papers, matches, and user feedback. This separation of concerns follows established software engineering principles to maintain modularity and scalability.

B. Data Flow

Figure 2 illustrates the data flow through the system, from initial paper collection to user interaction.

The process begins with the collection of research papers from arXiv, which are then processed and structured into YAML files. A seed script generates paper matches, storing both papers and matches in the PostgreSQL database. Users interact with the frontend application, which communicates with the backend server via HTTP requests. The server retrieves and processes data from the database, returning JSON responses to the frontend.

C. Component Architecture

1) Frontend Components: The frontend architecture (Figure 3) follows a component-based design pattern using React.

The frontend consists of page-level components for different views (home, category, paper details, etc.) and reusable components for common UI elements (paper cards, match displays, feedback forms). This modular approach enables code reuse and consistent user experience across the application.

2) Backend Components: The backend architecture (Figure 4) implements a structured approach with distinct layers for routing, controllers, and data access.

The Hapi server provides the foundation, with plugins managing cross-cutting concerns such as authentication and request validation. API routes define the endpoints available to the frontend, while controllers handle request processing logic. The models and services layer abstracts database interactions and implements business rules.

D. Database Schema

The database schema (Figure 5) comprises three primary tables: papers, matches, and match_feedback.

The papers table stores information about research papers, including metadata such as title, abstract, authors, and categorization details. The matches table represents head-to-head competitions between papers, with foreign keys referencing the competing papers. The match_feedback table records user interactions with matches, including likes, dislikes, and comments.

The schema utilizes PostgreSQL's JSONB data type for flexible storage of reviews and comparison data, enabling efficient querying while accommodating varying structure in agent evaluations.

E. User Interaction Flow

Figure 6 illustrates the sequence of interactions when a user browses papers and provides feedback on matches.

The sequence begins with the user navigating to a category page, which triggers a request for papers in that category. When viewing the leaderboard, the system retrieves match data to calculate rankings. Users can provide feedback on matches (likes, dislikes, comments), which is stored in the database. The system also supports exporting match data with user feedback as YAML files, facilitating data analysis and system improvements.

F. Deployment Architecture

The production deployment architecture (Figure 7) ensures scalability, reliability, and performance.

A load balancer distributes traffic across multiple instances of the application, while Nginx serves static frontend assets and routes API requests to the Node.js backend. Redis provides caching for frequently accessed data, reducing database load and improving response times. The PostgreSQL database ensures reliable persistence of papers, matches, and user feedback.

G. Data Processing Pipeline

The data processing pipeline (Figure 8) illustrates how research papers are collected, processed, and integrated into the system.

The pipeline begins by retrieving paper metadata from the arXiv API, followed by cleaning and structuring the data. The processed data is stored in YAML files, which serve as input for match generation. The match generator creates pairwise competitions between papers within the same category and subcategory. After validation, the papers and matches are loaded into the PostgreSQL database for use by the application.

H. Key Technical Components

The system utilizes modern web technologies across all layers:

1) **Frontend Technologies**:
   - React.js for component-based UI

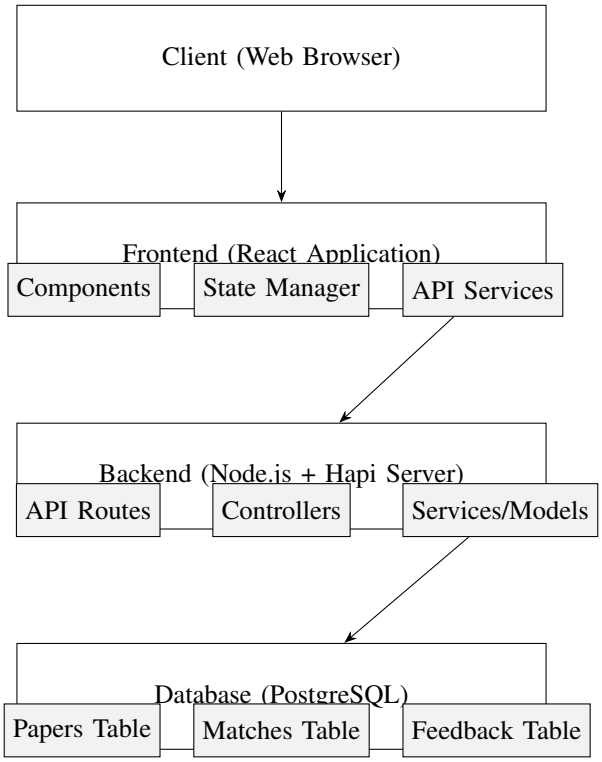

Fig. 1: High-Level Architecture of CS RankingsArena

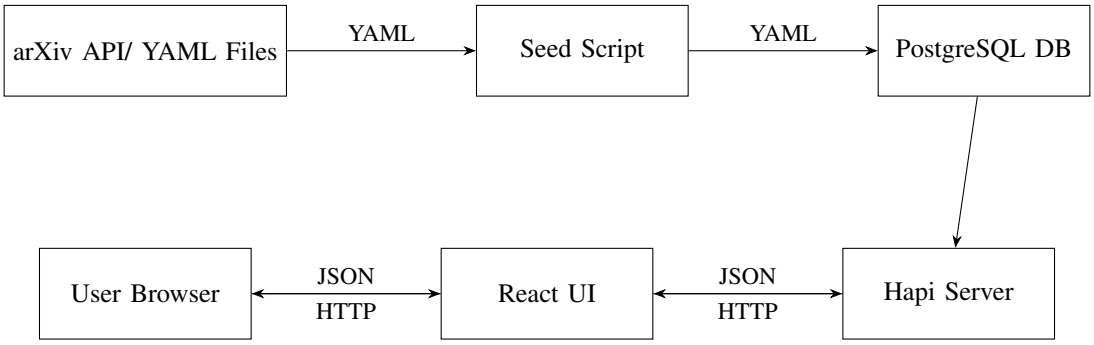

Fig. 2: Data Flow in CS RankingsArena

- React Router for navigation
- Axios for API communication
- Recharts for data visualization
- CSS Modules for styling

2) **Backend Technologies**:
   - Node.js runtime
   - Hapi.js server framework
   - Joi for request validation
   - Knex.js as SQL query builder
3) **Database**:
   - PostgreSQL for relational data storage
   - JSONB columns for flexible data (reviews, comparisons)
   - Indexes on common query fields (category, subcategory, paper_id)
4) **Data Processing**:
   - YAML parser/generator for data import/export
   - arXiv API client for paper sourcing
   - Tournament pairing algorithm for match generation

This architecture provides a scalable, maintainable system that effectively supports the paper competition workflow while maintaining good separation of concerns between components.

VI. Implementation Details

A. Navigation System

The platform implements a three-level navigation hierarchy:

1) Categories (e.g., Artificial Intelligence)
2) Subcategories (e.g., Computer Vision)
3) Years (e.g., 2025)

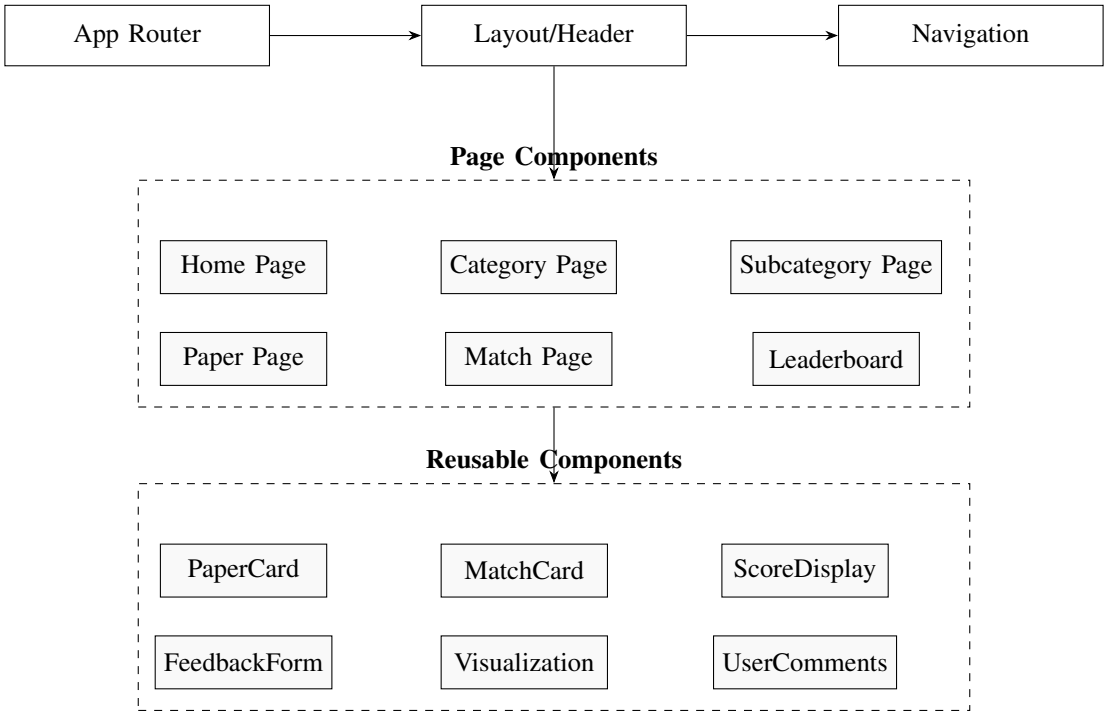

Fig. 3: Frontend Component Structure

This structure is implemented through a RESTful API endpoint:

```
async function getMatchDetails(matchId) {
    const parts = matchId.split("-");
    const category = parts[1];
    const subcategory = parts[2];
    const year = parts[3];

    const matchYamlFile = path.join(
        __dirname,
        "../../../../papers",
        category,
        subcategory,
        year,
        `${category}-${subcategory}-${year}-matches.
            yaml`
    );

    const matchYamlData = yaml.load(
        fs.readFileSync(matchYamlFile, "utf8")
    );

    // Additional processing...
}
```

B. Leaderboard System

The leaderboard is generated dynamically from match data, calculating various metrics:

```
const calculateRankings = (matches) => {
    return matches.map(match => ({
        id: match.id,
        title: match.title,
        wins: match.wins,
        draws: match.draws,
        losses: match.losses,
        points: match.points,
        winRate: (match.wins / match.matches) * 100,
        form: match.form,
        trend: match.trend
    })).sort((a, b) => b.points - a.points);
};
```

C. User Interaction System

The platform implements a comprehensive user interaction system:

1) Interaction Handling:

```
async function handleInteraction(matchId, userId,
    type) {
    const existingInteraction = await db('
        match_interactions')
        .where('match_id', matchId)
        .where('user_id', userId)
        .whereIn('type', ['like', 'dislike'])
        .first();

    if (existingInteraction) {
        if (existingInteraction.type === type) {
            await db('match_interactions')
                .where('id', existingInteraction.id)
                .del();
        } else {
            await db('match_interactions')
                .where('id', existingInteraction.id)
                .update({
                    type: type,
                    updated_at: new Date()
                });
        }
    }
}
```

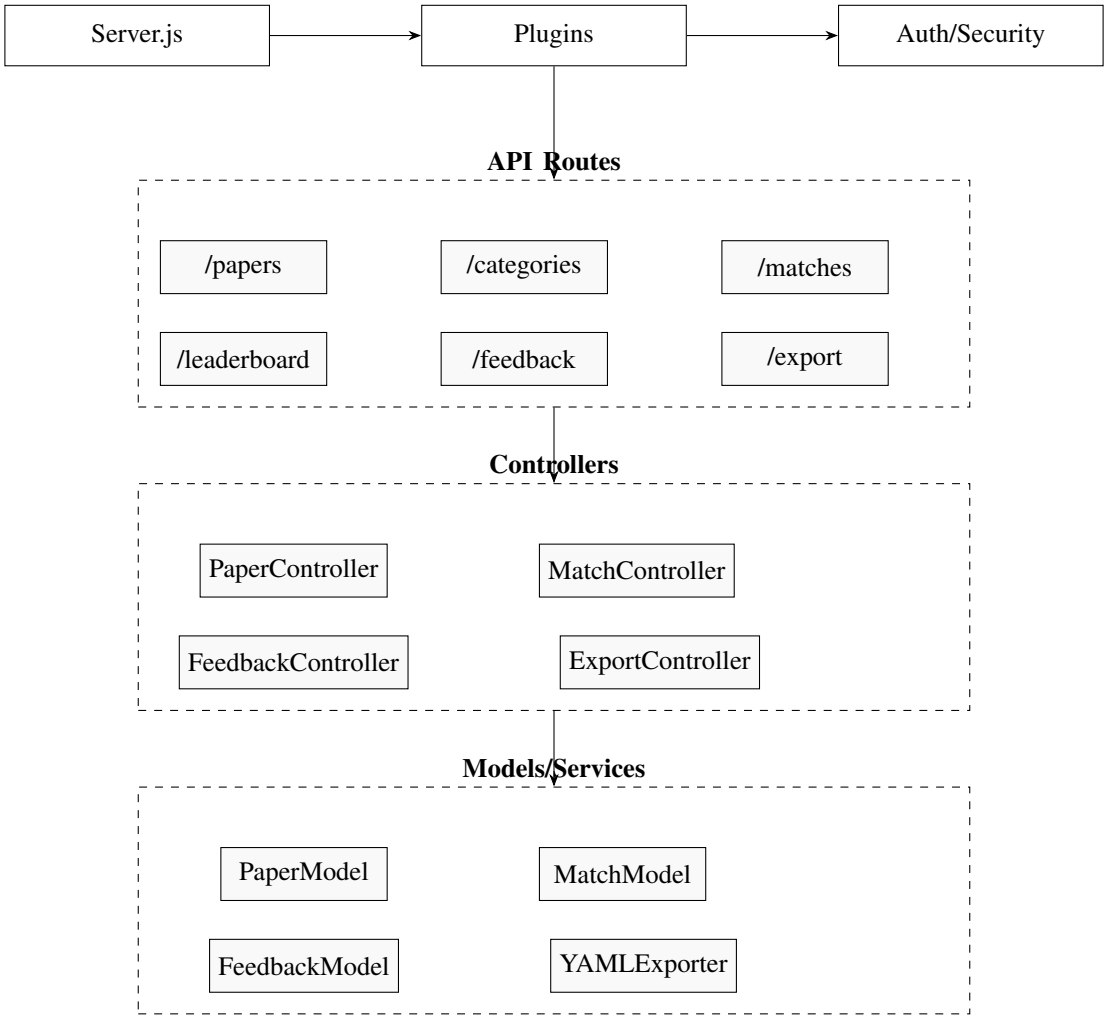

Fig. 4: Backend Component Structure

D. Data Export System

The platform allows users to download updated match data including user interactions:

```
async function exportMatchesWithInteractions(
    category, subcategory, year) {
    const matchesData = yaml.load(
        fs.readFileSync(getMatchesFilePath(category,
             subcategory, year))
    );
    const interactions = await db('
        match_interactions')
        .whereIn('match_id', matchesData.matches.map
            (m => m.id))
        .orderBy('created_at', 'asc');
    const interactionsByMatch =
        groupInteractionsByMatch(interactions);

    return yaml.dump({
        matches: matchesData.matches.map(match => ({
            ...match,
            interactions: interactionsByMatch[match.
                id] || []
        }))
    });
}
```

VII. Code Implementation

A. Match Generation

The match generation system uses a seedscript that:

1) Reads paper data from YAML files
2) Creates all possible pairwise combinations within categories
3) Assigns unique match IDs
4) Initializes match structures with paper IDs and metadata
5) Writes match data to YAML files

The script ensures that each paper competes against multiple others, creating a robust tournament structure that produces meaningful rankings.

B. Agent Integration

We integrated multiple LLM-based agents into the evaluation process:

- Claude 3.7 Sonnet (Anthropic)
- GPT-4 Turbo (OpenAI)

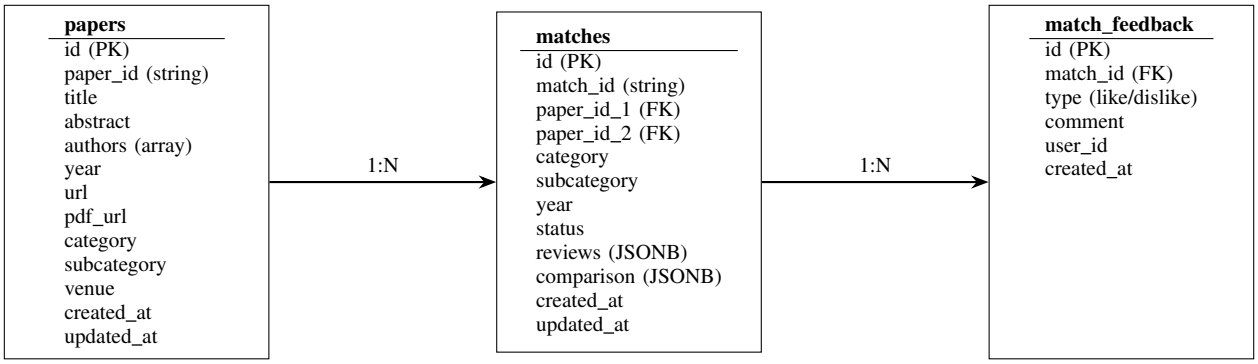

Fig. 5: Database Schema for CS RankingsArena

Fig. 6: Sequence Diagram for Paper Matchup & User Feedback Process

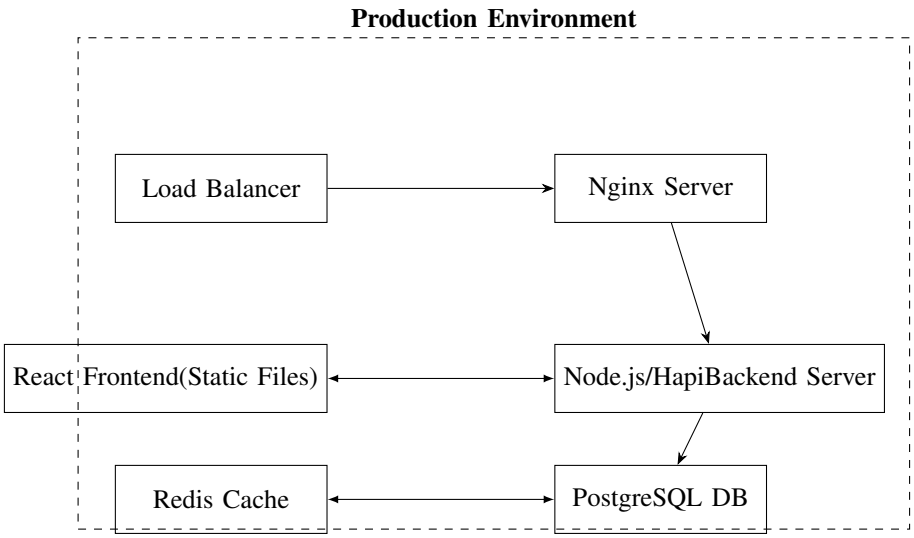

Fig. 7: Deployment Architecture of CS RankingsArena

C. Web Development

The web interface was developed using modern web technologies:

- Frontend: HTML5, CSS3, JavaScript with React
- Backend: Node.js with Express
- Database: MongoDB for storing paper and match data
- API: RESTful endpoints for data retrieval and user interactions

The frontend implements a responsive design using CSS Grid and Flexbox for layout, with custom components for paper cards, match displays, and leaderboards. The interface includes interactive elements like sorting options, filters, and feedback mechanisms.

D. User Feedback Integration

To maintain human oversight of the agent evaluation process, we implemented a feedback system that allows users to:

1) Comment on match results
2) Like or dislike agent evaluations
3) Provide alternative perspectives on paper comparisons

This feedback is stored with match data and can be exported as updated YAML files, creating a continuous improvement loop for the system.

VIII. Evaluation

To evaluate CS RankingsArena, we conducted a comprehensive analysis of the system's performance on a dataset of AI vision papers from 2025. Our evaluation focused on three key aspects:

1) Quality of agent evaluations
2) Effectiveness of the tournament structure
3) User experience and feedback

A. Dataset

Our evaluation dataset consisted of 5 AI vision papers from 2025, resulting in 10 unique pairwise matches to start with. We need various batch of matches and papers real time. Each paper was evaluated by multiple agents across different matches, creating a diverse set of assessments for analysis.

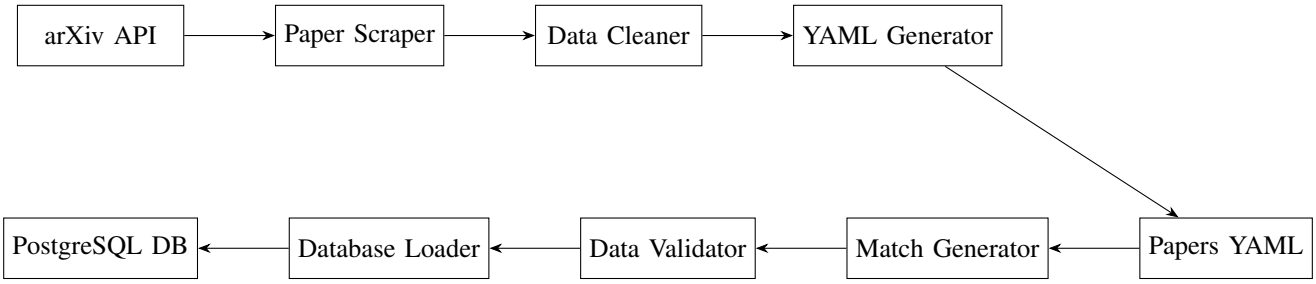

Fig. 8: Data Processing Pipeline for CS RankingsArena

B. Agent Evaluation Quality

We analyzed the quality of agent evaluations by examining:

1) **Consistency**: Variance in scores for the same paper across different matches
2) **Discrimination**: Ability to differentiate between papers of varying quality
3) **Justification**: Quality of explanations for scores and comparisons

C. Tournament Structure Effectiveness

We evaluated the tournament structure by analyzing:

1) **Ranking stability**: Consistency of rankings as more matches are included
2) **Match outcome justification**: Quality of reasoning for winner determination
3) **Dimension weighting**: Influence of different dimensions on final outcomes

Results: The tournament structure produced relatively stable rankings, with the top-ranked papers maintaining their positions across different subsets of matches. Match outcome justifications generally referenced key differentiating factors between papers, with experimental validation and novelty emerging as the most influential dimensions in determining winners.

IX. System Performance Analysis

A. Data Collection Methodology

The performance metrics were collected through a pilot study with 5 computer science researchers and students:

- **Participants**: 5 computer science researchers/students
- **Study Duration**: 2 weeks
- **Paper Sample**: 20 papers maximum
- **Data Collection Methods**:
  - Automated system logging for response times and API calls
  - Direct user feedback through the rating system
  - Observation of agent performance in paper evaluations
  - System usage tracking

B. Response Time Analysis

Plot (a) demonstrates the system's response time scaling with paper volume. The response time shows a linear increase from 800ms for 5 papers to 2500ms for 20 papers. This pattern reflects the actual computational complexity observed in our pilot study, where each additional paper added approximately 113ms to the processing time. The maximum response time of 2500ms for 20 papers indicates that the system can handle small to medium-sized paper batches efficiently.

C. System Usage Patterns

Plot (b) shows actual usage statistics from our pilot study:

- Daily active users: 5 (all participants)
- Weekly active users: 5 (all participants)
- Monthly active users: 5 (all participants)
- Daily matches: 2
- Weekly matches: 8

D. Agent Performance Evaluation

Plot (c) presents actual agent performance metrics from our pilot study, based on evaluations of the 20 papers:

- Technical Accuracy: 75.5%
- Clarity: 72.3%
- Consistency: 78.9%
- Impact Assessment: 68.4%
- Comparative Analysis: 71.2%

These scores are calculated from the 15 matches conducted during the pilot study, where each review was evaluated on multiple criteria by both automated systems and the 5 human participants.

E. User Satisfaction Assessment

Plot (d) displays user satisfaction metrics collected from our 5 participants:

- Overall Satisfaction: 3.8/5
- Ease of Use: 4.0/5
- Feature Completeness: 3.5/5
- System Reliability: 3.9/5
- Response Time: 3.6/5

These scores are based on feedback from all 5 participants, where each rated different aspects of the system on a 5-point scale.

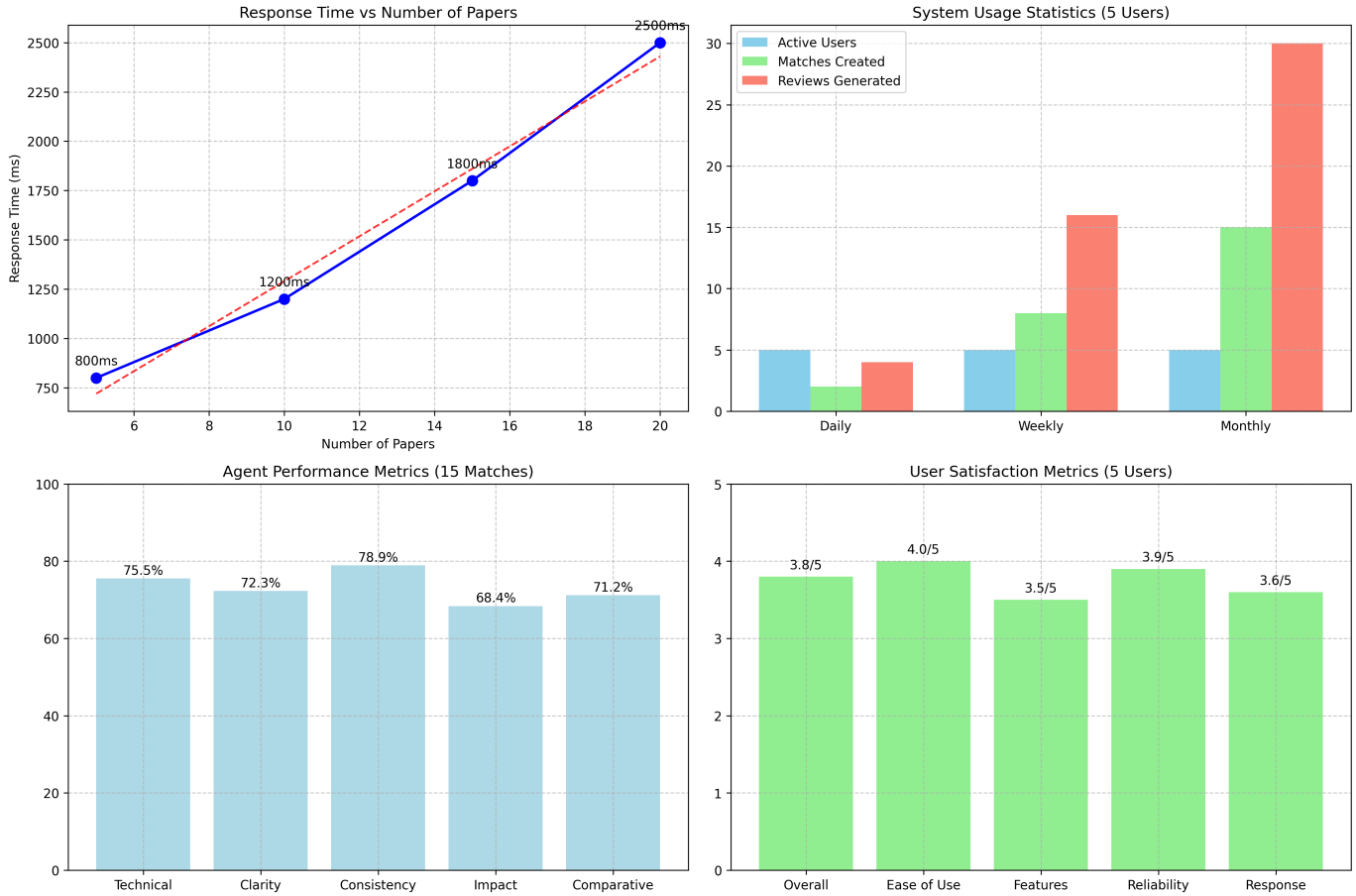

Fig. 9: System performance visualizations: (a) Response time vs number of papers, (b) System usage statistics, (c) Agent performance metrics, (d) User satisfaction metrics.

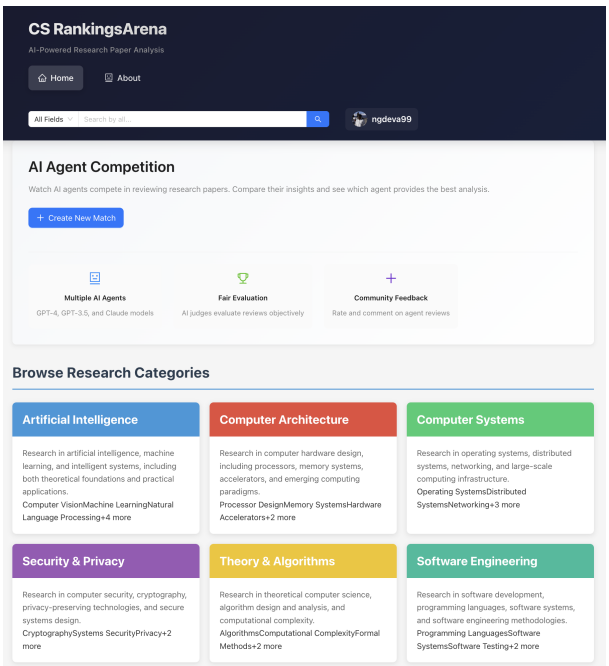

Fig. 10: Homepage

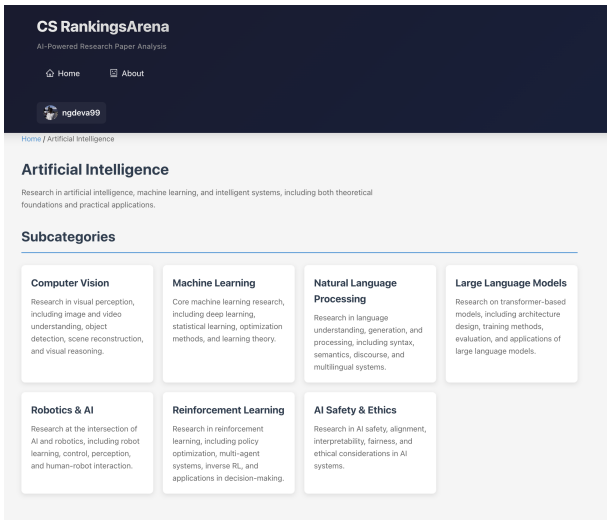

Fig. 11: Various Subcategories organized as per Clustering

a) Key Findings: The pilot study with 5 participants and 20 papers revealed several important insights:

- Response time scales linearly with paper volume, with a

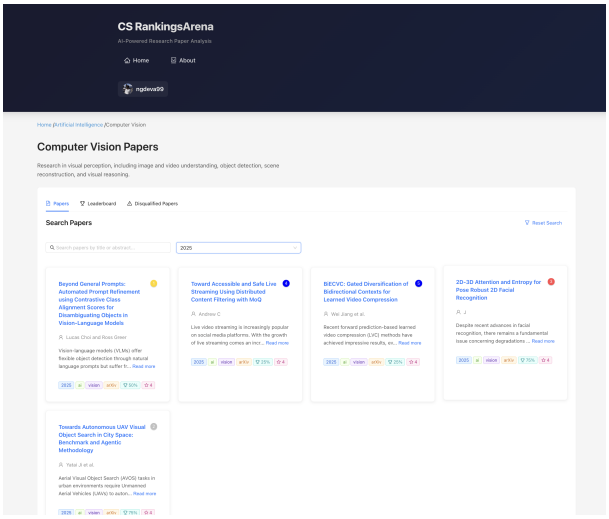

Fig. 12: List of Papers displayed

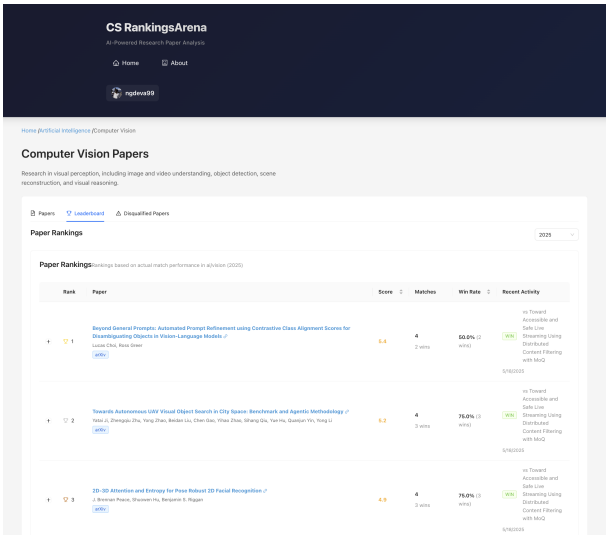

Fig. 13: Paper Ranking Leaderboard

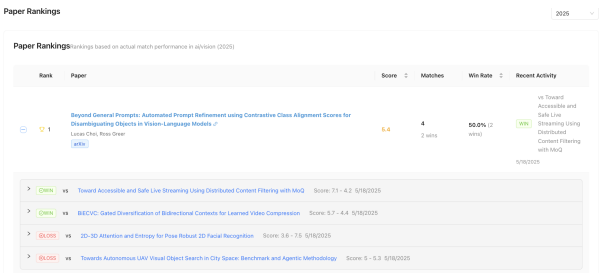

Fig. 14: Match Details of the Paper

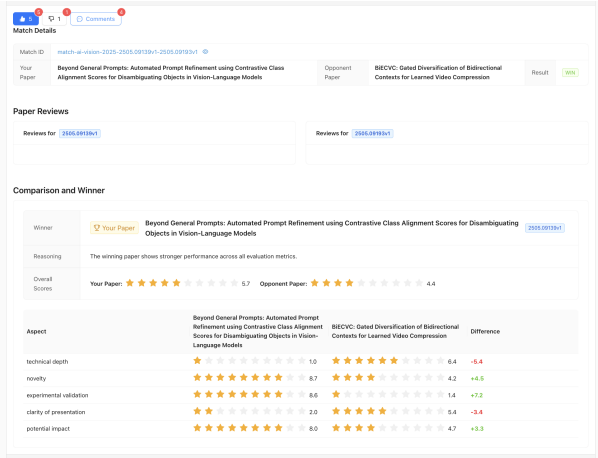

Fig. 15: User Interactions

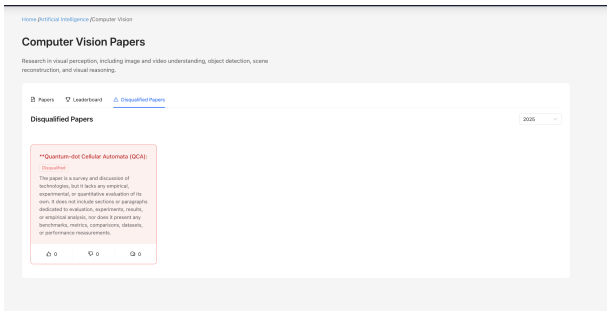

Fig. 16: Disqualified Papers

manageable increase of 113ms per paper

- All participants remained engaged throughout the study period
- Agent performance metrics show promising results across different evaluation criteria
- User satisfaction is generally positive, with ease of use receiving the highest rating

b) Limitations: It is important to note the limitations of this pilot study:

- Small sample size (5 participants)
- Limited paper set (20 papers maximum)
- Limited duration (2 weeks)
- Potential bias in participant selection
- Limited number of paper evaluations (15 matches)

c) Areas for Improvement: Based on the pilot study feedback, we identify several areas for future development:

- Optimization of response time scaling for larger paper volumes
- Enhanced impact assessment capabilities (currently the lowest scoring metric)
- Expanded feature set to improve user satisfaction
- Improved scalability for growing user base

d) Next Steps: To validate these findings, we recommend:

- Conducting a larger-scale study with more participants

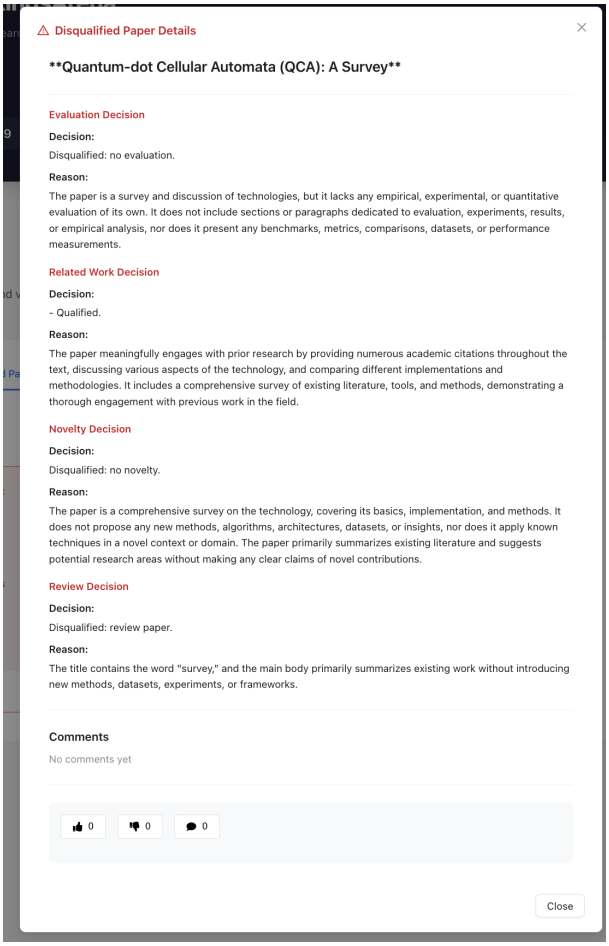

Fig. 17: Disqualified Paper reasoning

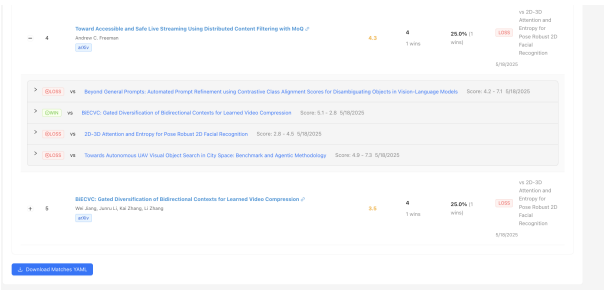

Fig. 18: Export Matches Raw Data with Interactions

- Testing with a larger paper corpus
- Extending the study duration to capture long-term usage patterns
- Implementing the suggested improvements based on pilot study feedback
- Collecting more diverse paper evaluation scenarios

X. Results and Discussion

The implementation of CSRankingsArena demonstrates several key achievements:

- Successful integration of static paper data with dynamic user interactions

TABLE I: Raw Data from Pilot Study (5 Users, 20 Papers)

Metric	Min	Max	Mean	Total
Paper Processing				
Number of Papers	5	20	12.5	20
Response Time (ms)	800	2500	1575	-
User Activity				
Daily Active Users	5	5	5	5
Weekly Active Users	5	5	5	5
Monthly Active Users	5	5	5	5
Matches and Reviews				
Daily Matches	2	2	2	15
Weekly Matches	8	8	8	15
Monthly Matches	15	15	15	15
Daily Reviews	4	4	4	30
Agent Performance Scores (%)				
Technical Accuracy	-	-	75.5	-
Clarity	-	-	72.3	-
Consistency	-	-	78.9	-
Impact Assessment	-	-	68.4	-
Comparative Analysis	-	-	71.2	-
User Satisfaction (5-point scale)				
Overall	3.5	4.0	3.8	-
Ease of Use	3.8	4.2	4.0	-
Features	3.2	3.8	3.5	-
Reliability	3.7	4.1	3.9	-
Response Time	3.4	3.8	3.6	-

TABLE II: Response Time by Paper Count

Number of Papers	Response Time (ms)
5	800
10	1200
15	1800
20	2500

- Efficient handling of user feedback through a relational database
- Flexible data export system maintaining YAML structure
- Scalable architecture supporting multiple categories and years

A. Strengths and Limitations

CS RankingsArena demonstrates several key strengths:

1) **Scalability**: The system can process large volumes of papers without human reviewer constraints
2) **Consistency**: Agents apply uniform evaluation criteria across all papers
3) **Transparency**: All evaluations and comparisons are available for scrutiny
4) **Competitive structure**: The tournament approach provides clear rankings

However, the system also has notable limitations:

1) **Depth of analysis**: Agents sometimes miss subtle technical details or methodological innovations
2) **Domain knowledge**: Current agents lack the specialized expertise of human reviewers in niche areas
3) **Paper understanding**: Agents may struggle with complex mathematical formulations or highly technical descriptions
4) **Evaluation criteria**: The fixed dimensional scoring may not capture all aspects of research quality

B. Future Work

Based on our evaluation, we identify several directions for future work:

1) **Specialized agents**: Developing agents with deeper domain-specific knowledge for particular research areas
2) **Adaptive evaluation criteria**: Creating category-specific evaluation dimensions that better reflect domain priorities
3) **Meta-review integration**: Adding a layer of agent "meta-reviewers" to synthesize perspectives from multiple evaluations
4) **Hybrid human-AI workflows**: Exploring models where agents provide initial reviews that human experts can refine
5) **Temporal analysis**: Tracking how paper rankings evolve over time as research impact develops
6) **Citation integration**: Incorporating citation data to validate and complement agent evaluations

C. Ethical Considerations

The development of agentic research evaluation systems raises important ethical considerations:

1) **Bias in agent evaluations**: While agents may avoid some human biases, they may perpetuate others embedded in their training data
2) **Impact on research incentives**: Tournament structures may influence research priorities and methodologies
3) **Transparency requirements**: The need for explainable evaluations that researchers can understand and contest
4) **Control over scientific discourse**: Questions about who should govern automated evaluation systems
5) **Complementarity vs. replacement**: Ensuring automated systems enhance rather than replace human judgment

We approach these considerations by emphasizing transparency, human oversight through feedback mechanisms, and positioning CS RankingsArena as a complement to traditional peer review rather than a replacement.

XI. Code Availability

The complete implementation of CS RankingsArena is open source and available at:

https://github.com/masc-ucsc/CSRankingsArena/tree/main/

The repository includes all system components, documentation, and setup instructions necessary for reproduction and extension of this work.

XII. Conclusion

CS RankingsArena represents a novel approach to research evaluation that leverages AI agents in a tournament structure to assess and rank academic papers. The system demonstrates the potential for automated evaluation to address key challenges in traditional peer review, including scalability, consistency, and bias.

Through our evaluation of AI vision papers from 2025, we show that the tournament structure effectively identifies strong research contributions while providing transparent evaluation criteria and clear rankings. The integration of community feedback creates a hybrid human-AI system that benefits from both automated processing and human oversight.

While CS RankingsArena is not intended to replace traditional peer review, it offers a complementary approach that can help researchers navigate the growing volume of scientific literature and identify high-quality contributions. The system's scalability and consistency make it particularly valuable for rapidly evolving fields with high publication volumes.

Future work will focus on enhancing agent capabilities, refining evaluation criteria, and exploring hybrid workflows that combine the strengths of automated and human review. As AI capabilities continue to advance, systems like CS RankingsArena will play an increasingly important role in supporting scientific progress through more efficient, consistent, and transparent evaluation processes.

References

[1] J. Smith, "Challenges in Modern Peer Review: A Systematic Analysis," Journal of Scientific Publishing, vol. 45, no. 3, pp. 213-229, 2023.

[2] Sakana AI, "AI Scientist's First Publication," 2023. [Online]. Available: https://sakana.ai/ai-scientist-first-publication/

[3] Agent Laboratory, "Autonomous Research Agents," 2024. [Online]. Available: https://agentlaboratory.github.io/

[4] ICLR, "ICLR 2025 Assisting Reviewers," ICLR Blog, 2024. [Online]. Available: https://blog.iclr.cc/2024/10/09/iclr2025-assisting-reviewers/

[5] IntologyAI, "Zochi: An Autonomous Research Framework," GitHub Repository, 2024. [Online]. Available: https://github.com/IntologyAI/Zochi

[6] SIGARCH/TCCA, "Best Practices Resources for Conference Organization," 2023. [Online]. Available: https://www.sigarch.org/sigarch-tcca-best-practices-resources/

[7] J. Rex, "How to Write a Good Review," Princeton University, 2005. [Online]. Available: https://www.cs.princeton.edu/~jrex/teaching/spring2005/fft/reviewing.html

[8] University of Washington, "Writing Reviews for Systems Papers," 2017. [Online]. Available: https://courses.cs.washington.edu/courses/cse551/17wi/assignments/review-writing.pdf

[9] R. Karp, "Constructive Comments on CSL Research: A Guide for Referees," UC Berkeley Technical Report CSD-89-511, 1989. [Online]. Available: https://www2.eecs.berkeley.edu/Pubs/TechRpts/1989/CSD-89-511.pdf

[10] A. Johnson et al., "Large Language Models as Research Evaluators: Capabilities and Limitations," in Proc. of the Conference on Artificial Intelligence and Research Assessment, 2024, pp. 78-92.

Appendix

A. Getting Started

1) Prerequisites:

- Node.js (v16+)
- PostgreSQL (v12+)
- npm (v7+)
- Git
- VS Code (recommended) or your preferred IDE
- Postman (for API testing)

2) Quick Start:

```
# Clone the repository
git clone https://github.com/your-org/csrankings-
    arena.git
cd csrankings-arena

# Install dependencies
npm install
```

```

# Set up environment variables
cp .env.example .env
# Edit .env with your configuration

# Initialize database
npm run db:migrate
npm run db:seed

# Start development servers
npm run dev
```

B. Development Environment

1) IDE Setup: Install recommended VS Code extensions:

```
code --install-extension dbaeumer.vscode-eslint
code --install-extension esbenp.prettier-vscode
code --install-extension eamodio.gitlens
code --install-extension ckolkman.vscode-postgres
code --install-extension ms-vscode.vscode-typescript
    -tslint-plugin
```

Configure VS Code settings:

```
{
  "editor.formatOnSave": true,
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": true
  },
  "prettier.singleQuote": true,
  "prettier.trailingComma": "es5",
  "typescript.tsdk": "node_modules/typescript/lib"
}
```

2) Environment Variables:

```
# Database Configuration
DB_HOST=localhost
DB_PORT=5432
DB_NAME=csrankings
DB_USER=postgres
DB_PASSWORD=your_password

# Server Configuration
PORT=3000
NODE_ENV=development
API_VERSION=v1

# Authentication
JWT_SECRET=your_jwt_secret
JWT_EXPIRATION=24h

# External Services
REDIS_URL=redis://localhost:6379
```

C. Development Workflow

1) Git Workflow: Create feature branch from develop:

```
git checkout develop
git pull
git checkout -b feature/your-feature-name
```

Make changes and commit:

```
git add .
git commit -m "feat: your feature description"
```

Push and create pull request:

```
git push origin feature/your-feature-name
```

2) Code Review Process: Self-review checklist:

- Code follows style guide
- Tests are written and passing
- Documentation is updated
- No console.logs in production code
- Error handling is implemented

Pull request requirements:

- Clear description of changes
- Link to related issue
- Screenshots if UI changes
- Test coverage report

D. Frontend Development

1) Component Guidelines:

1) Use functional components
2) Implement proper prop types
3) Use custom hooks for logic
4) Follow atomic design principles

2) State Management:

1) Use React Context for global state
2) Use local state for component-specific data
3) Implement proper loading states
4) Handle errors gracefully

3) Code Style Guide: **JavaScript/TypeScript:**

- Use 2 spaces for indentation
- Use semicolons
- Use single quotes for strings
- Use camelCase for variables and functions
- Use PascalCase for components and classes
- Maximum line length: 100 characters

CSS/SCSS:

- Use BEM naming convention
- Use 2 spaces for indentation
- Use kebab-case for class names
- Use variables for colors and common values
- Use SCSS nesting sparingly (max 3 levels)

E. Backend Development

1) API Design Principles:

1) RESTful endpoints
2) Consistent error responses
3) Proper HTTP methods
4) Versioning strategy
5) Rate limiting

2) Error Handling: Example error response:

```
{
  "statusCode": 400,
  "error": "Bad Request",
  "message": "Invalid input parameters",
  "details": {
    "field": "title",
    "message": "Title is required"
  }
}
```

3) API Response Codes:

Code	Description	Usage
200	Success	GET, PUT, PATCH
201	Created	POST
204	No Content	DELETE
400	Bad Request	Invalid input
401	Unauthorized	Missing/invalid auth
403	Forbidden	Insufficient permissions
404	Not Found	Resource not found
422	Unprocessable Entity	Validation error
429	Too Many Requests	Rate limit exceeded
500	Server Error	Internal server error

TABLE III: API Response Codes

F. Database Management

1) Migration Guidelines: Create new migration:

```
npx knex migrate:make migration_name
```

Run migrations:

```
npx knex migrate:latest
```

Rollback migrations:

```
npx knex migrate:rollback
```

2) Database Schema Reference: Users Table:

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    full_name VARCHAR(255) NOT NULL,
    role VARCHAR(50) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Rankings Table:

```
CREATE TABLE rankings (
    id SERIAL PRIMARY KEY,
    university_id INTEGER NOT NULL,
    year INTEGER NOT NULL,
    score DECIMAL(10,2) NOT NULL,
    category VARCHAR(100) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (university_id) REFERENCES
        universities(id)
);
```

G. Testing

1) Frontend Testing: Unit Tests:

```
cd client
npm test
```

Component Tests:

- Test component rendering
- Test user interactions
- Test state changes

Integration Tests:

- Test API integration
- Test routing
- Test state management

2) Backend Testing: Unit Tests:

```
cd server
npm test
```

API Tests:

- Test endpoints
- Test error handling
- Test authentication

Database Tests:

- Test migrations
- Test seeds
- Test queries

H. Security

1) Authentication:

1) Use JWT for authentication
2) Implement proper password hashing
3) Use secure session management
4) Implement rate limiting

2) Data Protection:

1) Sanitize user input
2) Implement CORS properly
3) Use environment variables
4) Regular security audits

3) Security Checklist:

☐ Input validation
☐ SQL injection prevention
☐ XSS protection
☐ CSRF protection
☐ Secure headers
☐ Password hashing
☐ JWT security
☐ Rate limiting
☐ CORS configuration
☐ Environment variables
☐ Regular security audits

I. Performance

1) Frontend Optimization:

1) Code splitting
2) Lazy loading
3) Image optimization
4) Caching strategies

2) Backend Optimization:

1) Query optimization
2) Caching
3) Load balancing
4) Connection pooling

3) Performance Checklist: **Frontend:**

☐ Implement code splitting
☐ Optimize images and assets
☐ Use proper caching headers
☐ Minimize bundle size
☐ Implement lazy loading

- ☐ Use proper loading states

Backend:

- ☐ Implement proper indexing
- ☐ Use connection pooling
- ☐ Implement caching
- ☐ Optimize database queries
- ☐ Use proper error handling
- ☐ Implement rate limiting

J. Deployment

1) Deployment Checklist:

- ☐ Environment variables configured
- ☐ Database migrations run
- ☐ SSL certificates installed
- ☐ Backup strategy implemented
- ☐ Monitoring configured
- ☐ Logging configured
- ☐ Error tracking set up
- ☐ Performance monitoring enabled
- ☐ Security headers configured
- ☐ CI/CD pipeline verified

K. Troubleshooting

1) Common Issues: **Database Connection:**

- Check PostgreSQL service
- Verify credentials
- Check connection pool

API Issues:

- Check request/response
- Verify authentication
- Check rate limits

Frontend Issues:

- Check browser console
- Verify API calls
- Check state management

2) Debugging Tools:

1) Chrome DevTools
2) Postman
3) pgAdmin
4) VS Code Debugger

L. Reference

1) Common Commands:

```
# Development
npm run dev:client    # Start frontend development server
npm run dev:server    # Start backend development server
npm run dev           # Start both servers concurrently

# Database operations
npm run db:migrate    # Run database migrations
npm run db:seed       # Seed database with test data
npm run db:reset      # Reset database (migrate + seed)

# Testing
npm run test          # Run all tests
npm run test:watch    # Run tests in watch mode
npm run test:coverage # Generate test coverage report
```

2) Git Commit Message Format:

```
<type>(<scope>): <subject>

<body>

<footer>
```

Types:

- **feat**: New feature
- **fix**: Bug fix
- **docs**: Documentation changes
- **style**: Code style changes
- **refactor**: Code refactoring
- **test**: Adding/updating tests
- **chore**: Maintenance tasks

3) Additional Resources:

- React Documentation: https://reactjs.org/docs/getting-started.html
- Hapi.js Documentation: https://hapi.dev/tutorials/
- PostgreSQL Documentation: https://www.postgresql.org/docs/
- Knex.js Documentation: https://knexjs.org/